

CMPS310 Software Engineering

Fall 2024

Lecture 8

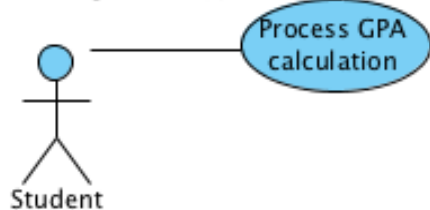
Design Sequence Diagram

Example : Design Sequence Diagram

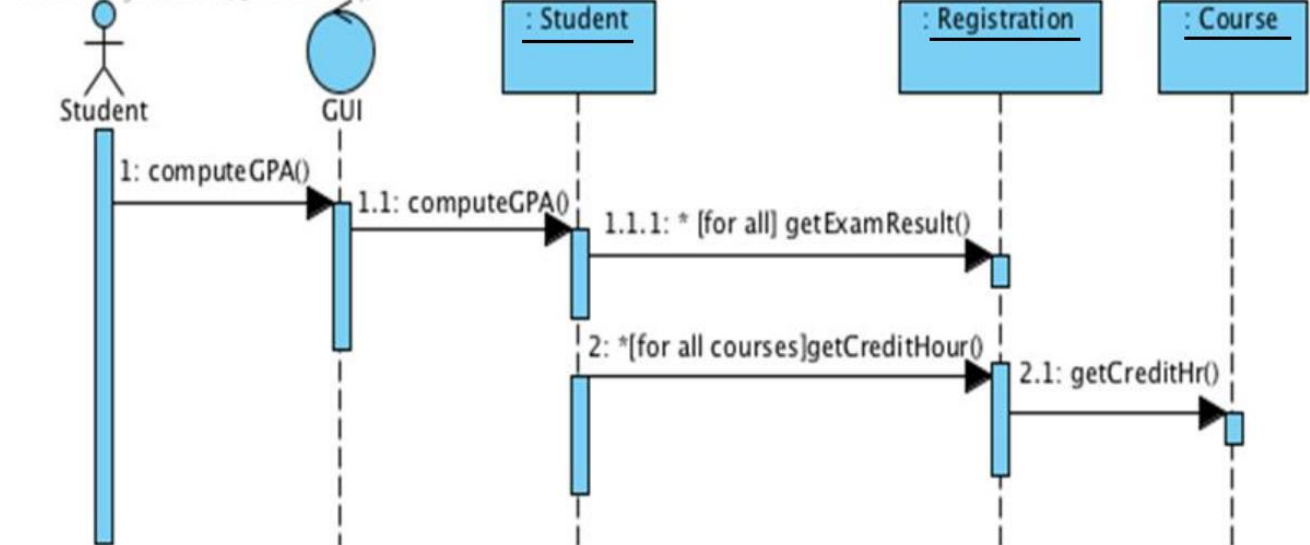
Actor Action	System Response
1. The Student wants to know her GPA	
2. The Student enters ID	3. Find exam result of each course that the student has registration
	4. Find credit hour of each course that the student has registration
	4. Compute GPA
	5. Store GPA
	6. Display GPA



Visual Paradigm Standard(Qatar University)



Visual Paradigm Standard(Qatar University)



Sequence diagram

Objects Need to Collaborate

- **Objects**
 - useless unless they can collaborate with other objects to solve a problem
 - one object cannot carry out every responsibility on its own
- How do objects interact with each other?
 - They interact **through passing messages**
 - A **method call** is the most common type of message
 - An **interaction is a set of messages exchanged** among a set of objects in order to accomplish a specific goal

Elements of Design Sequence Diagrams

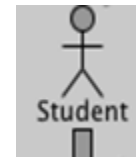
– Instances of classes

- Shown as boxes with the class and object identifier underlined



– Actors

- Use the stick-person symbol as in use case diagrams



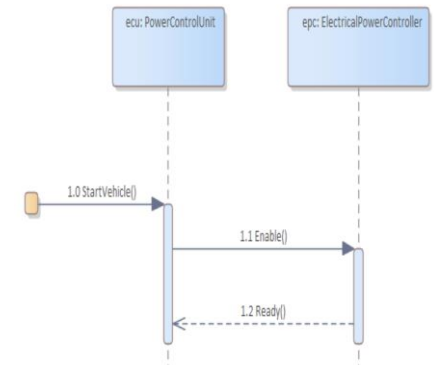
– Messages



- Shown as arrows from actor to object, or from object to object
- A message is the vehicle by which communication between objects is achieved
 - A *call* is the most common type of message
- The *return* of data as a result of a function call is also considered a message
- A message may result in a **change of state for the receiver of the message**
- The receipt of a message is considered an **instance of an event**

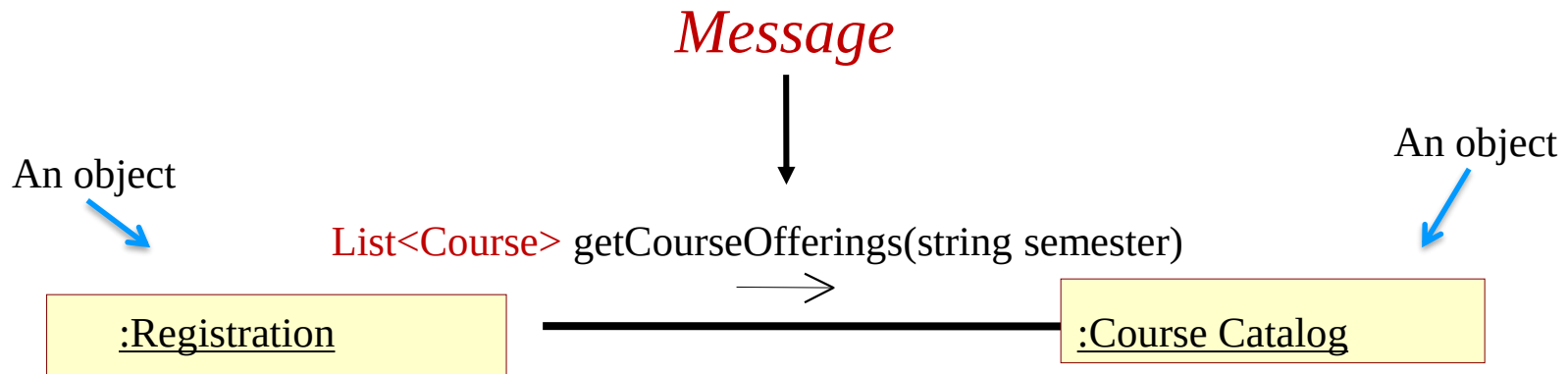
Design Sequence Diagrams

- A **design sequence diagram** shows the **sequence of messages exchanged** by the set of objects **performing a certain task**
 - Represents the white box representation (everything visible inside) of the system
 - Shows the **internal objects** of the system
 - Model which **object handles which message**
 - Shows **how messages propagate inside** the system boundary
 - The **objects are arranged horizontally** across the diagram
 - An **actor** that initiates the interaction is often **shown on the left**
 - The **vertical dimension** represents **time**
 - A vertical line, called a **lifeline**, is attached to each object or actor
 - **lifeline** becomes a broad box, called an **activation box** during the **live activation** period
 - A **message** is represented as an arrow between activation boxes of the sender and receiver
 - A message is numbered and labelled, can have an argument list and a return value



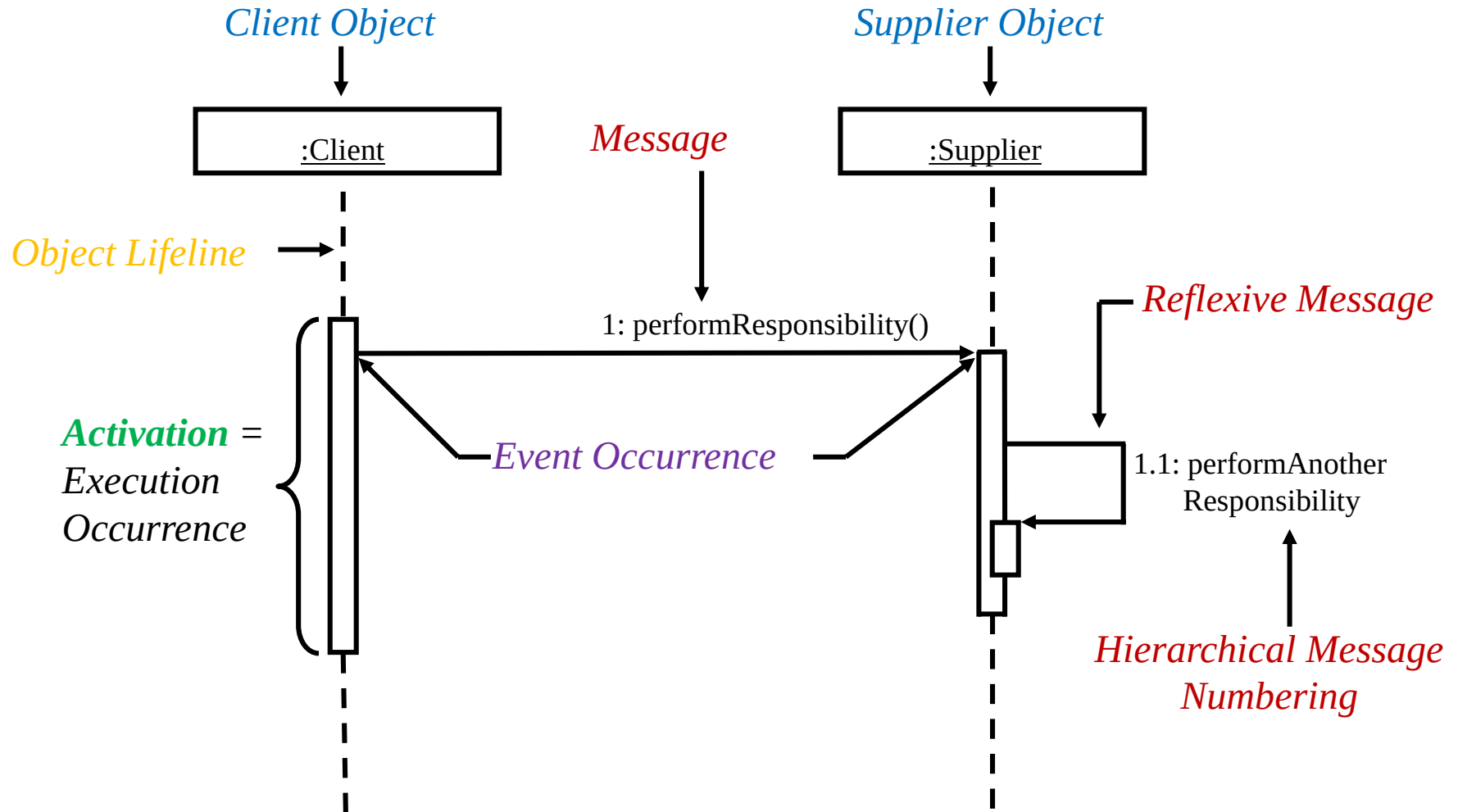
Objects Interact with Messages

- **Participant**
 - an object or entity that acts in the sequence diagram
 - sequence diagram starts with an unattached "found message" arrow
- **Message**
 - communication between participant objects
- **Axes**
 - **horizontal:** object/participant which is acting
 - **vertical:** time (down -> forward in time)
- **A message** (i.e., method call)
 - shows how one object asks another object to perform some activity



- When the activity has been executed, control is returned to the caller along with a return value

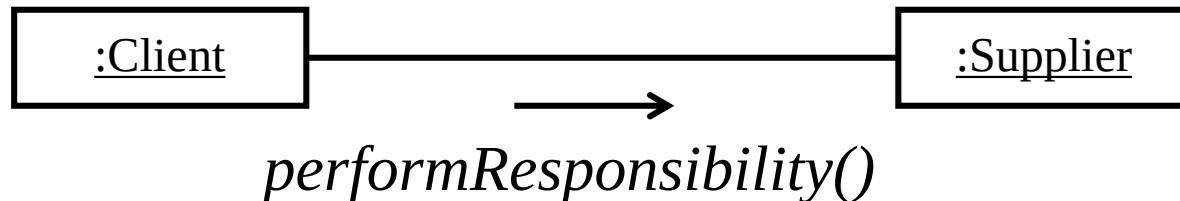
The Anatomy of Design Sequence Diagrams



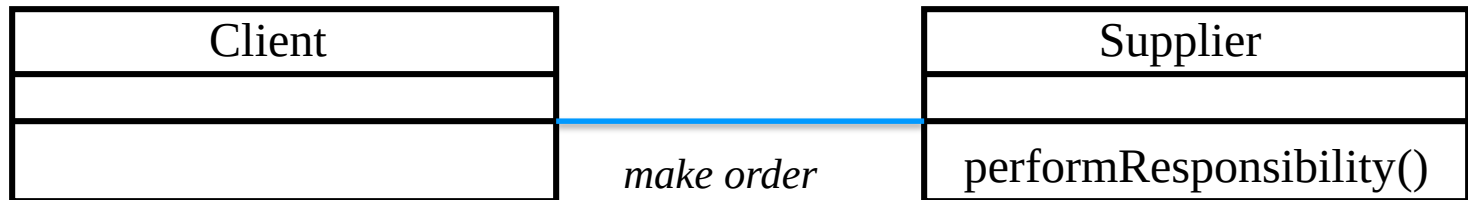
Relationship between Classes and Design Sequence Diagram

- *performResponsibility()* is a method defined in the **Supplier** class
- A Client object called this method of a Supplier object

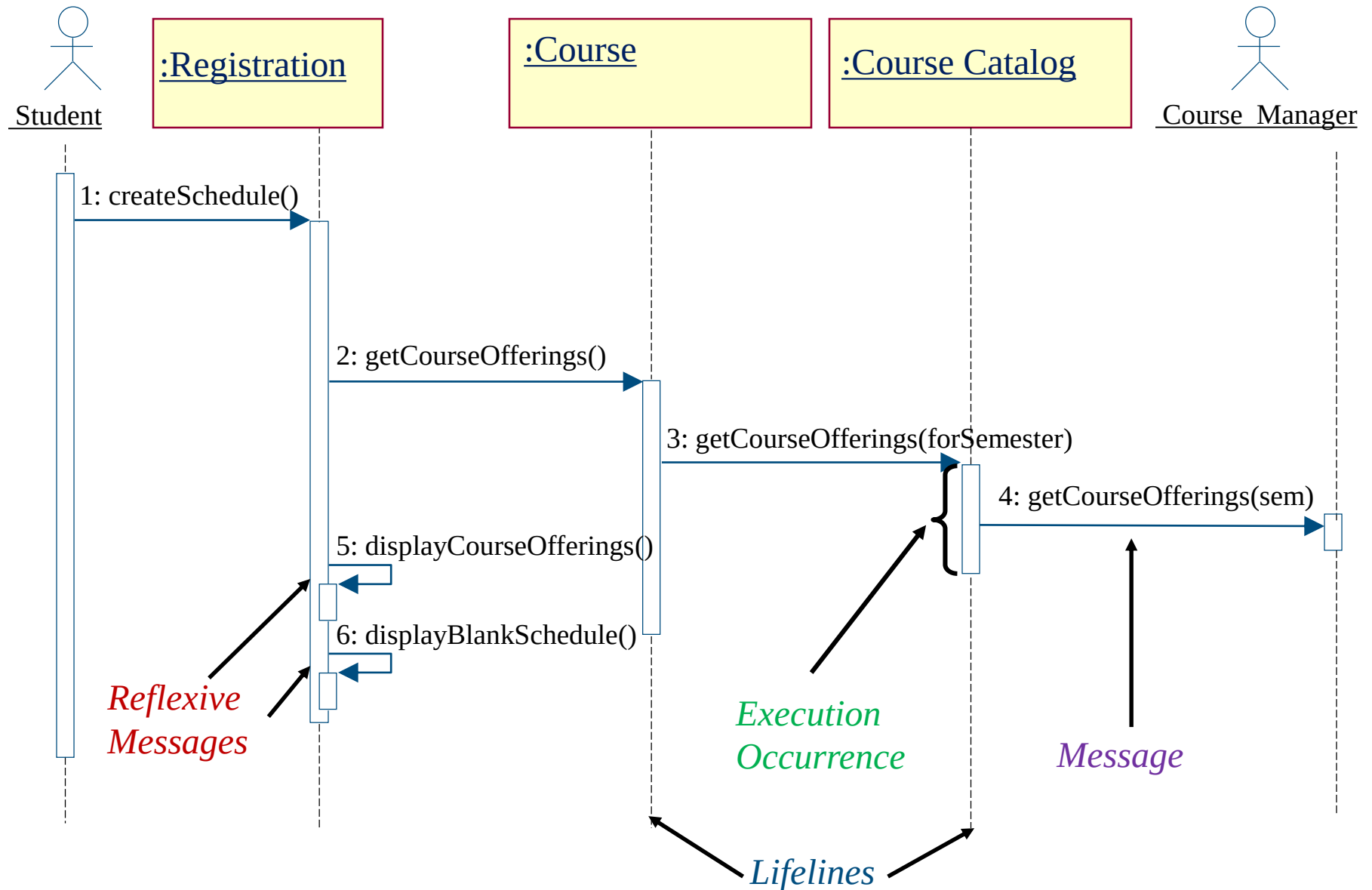
Sequence Diagram



Class Diagram

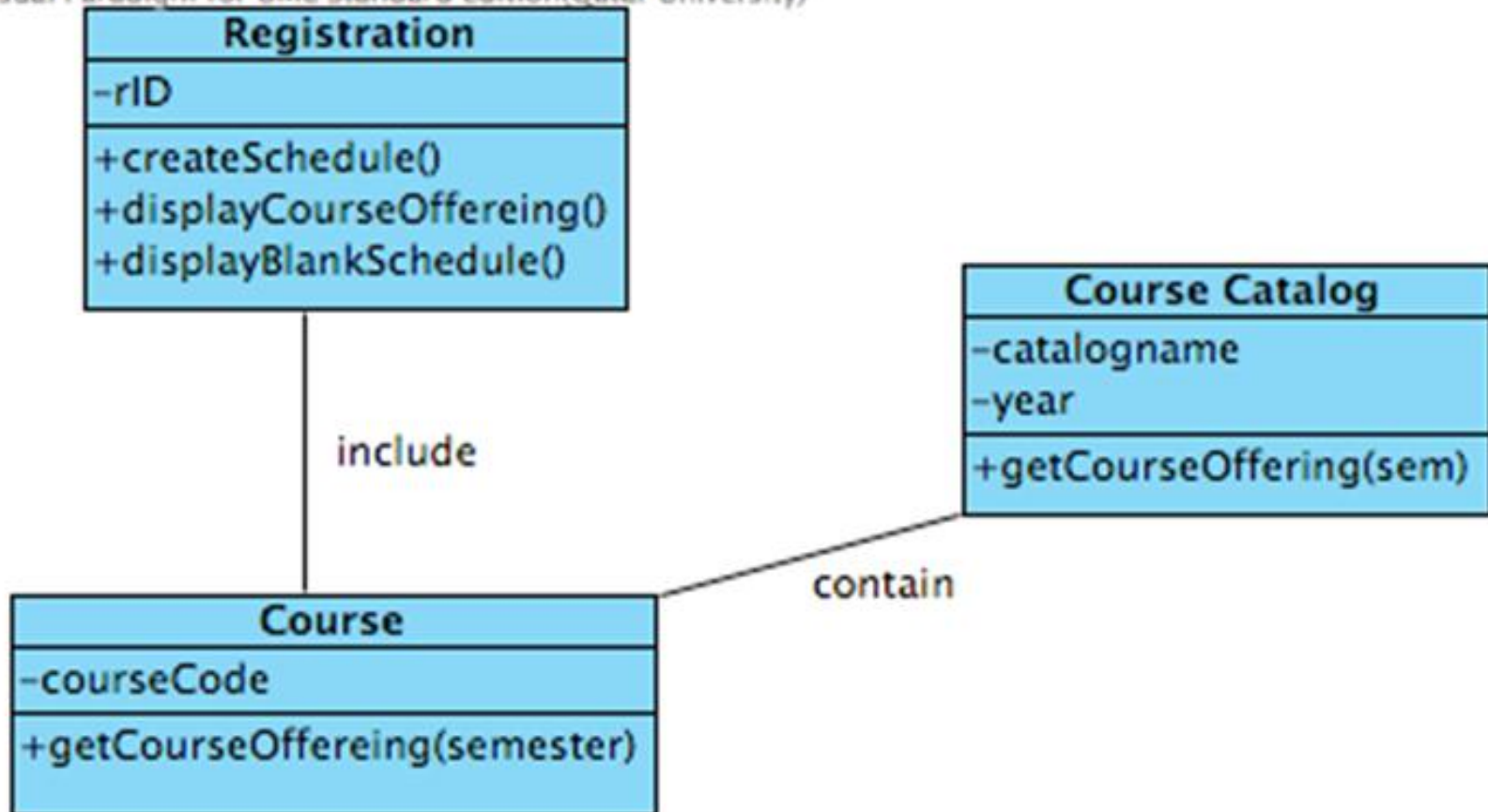


Sequence Diagram for Register for Courses Use Case



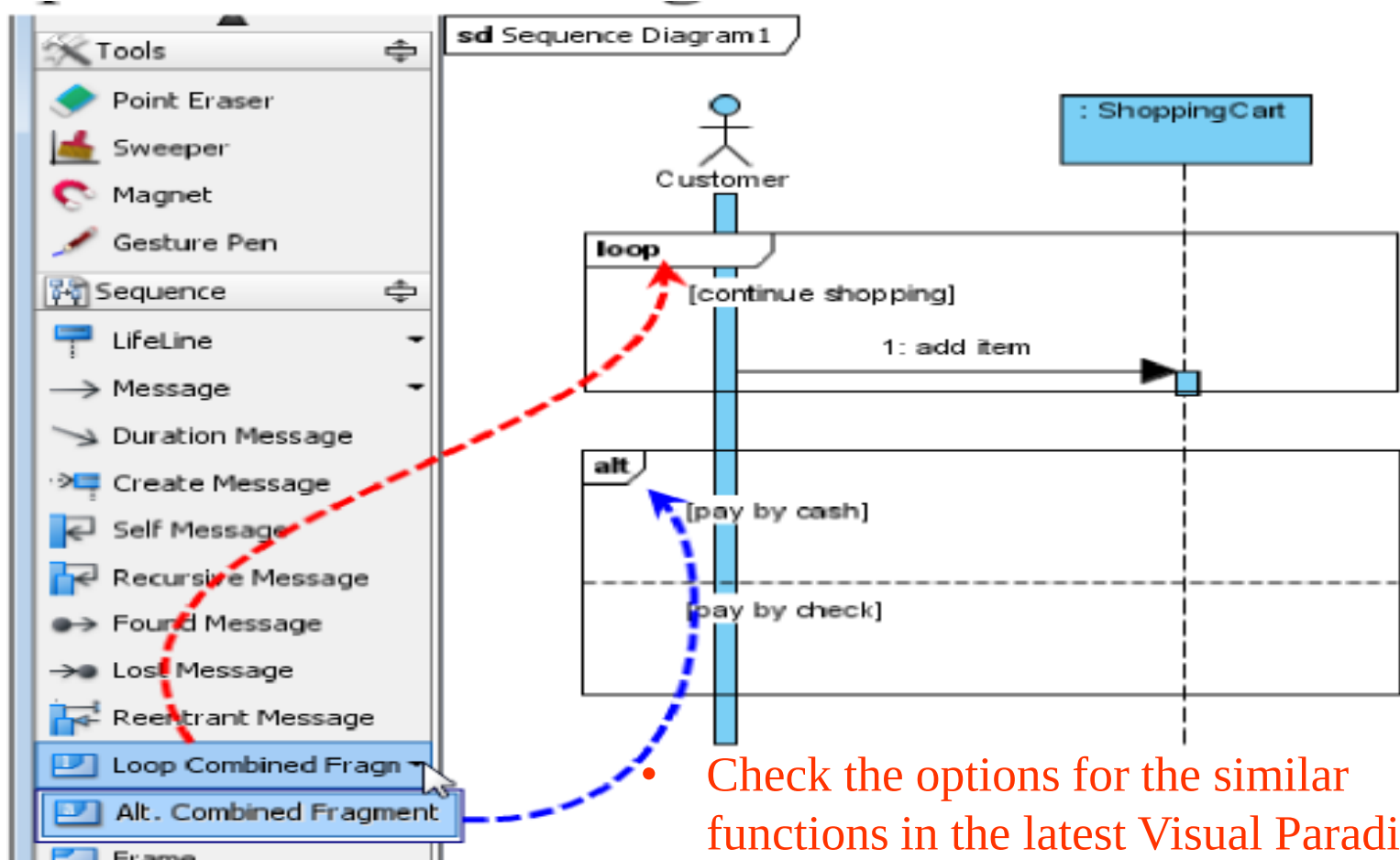
Class Diagram of The Previous Diagram

Visual Paradigm for UML Standard Edition(Qatar University)



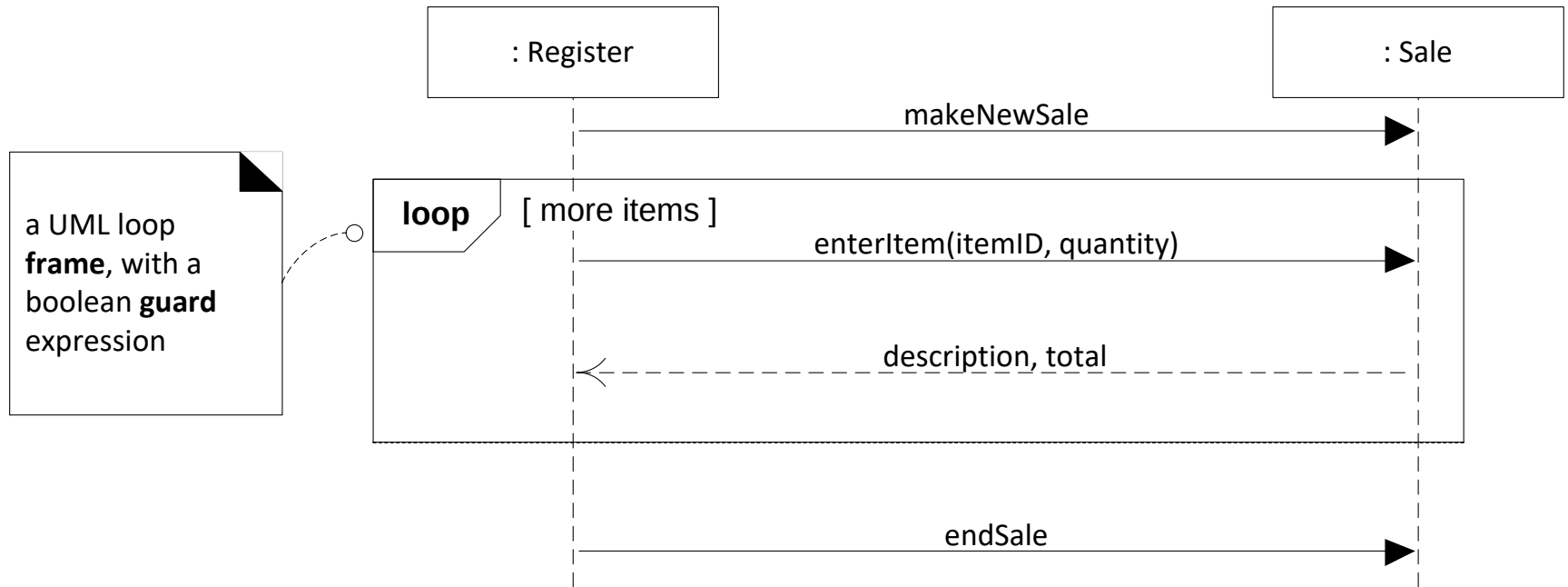
Example of using Alt and Loop

- Alt = If..Then
- Loop = While do, For loop, etc.



- Check the options for the similar functions in the latest Visual Paradigm

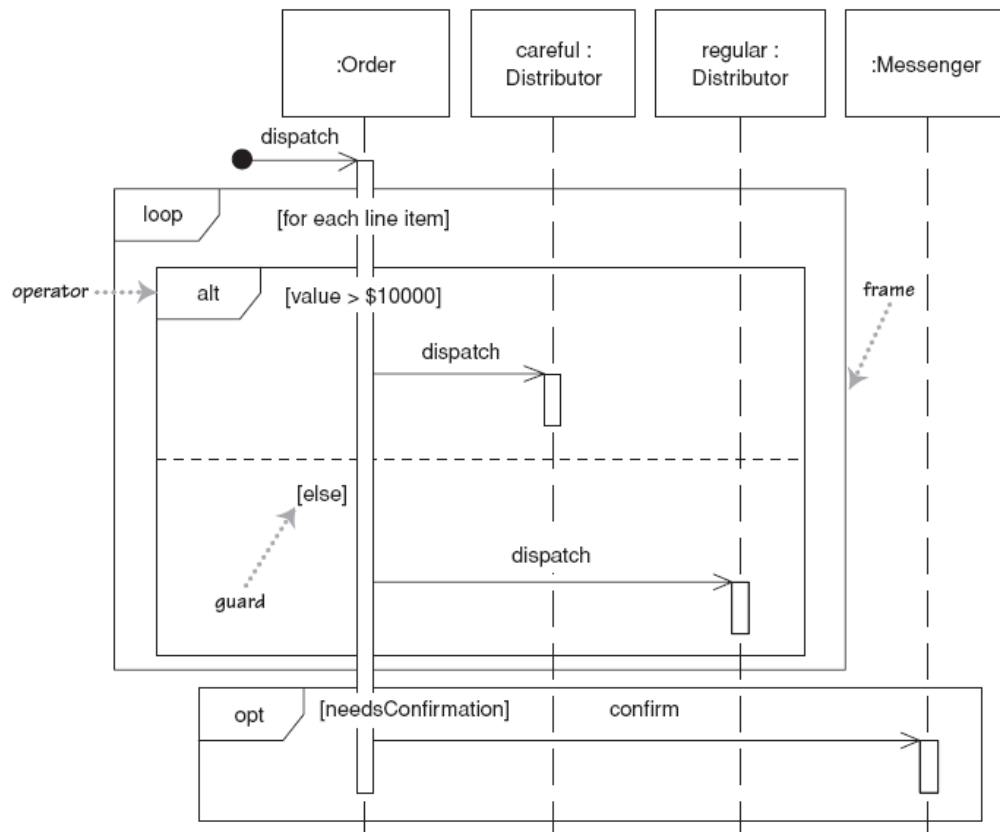
Design Sequence Diagram with **loop**



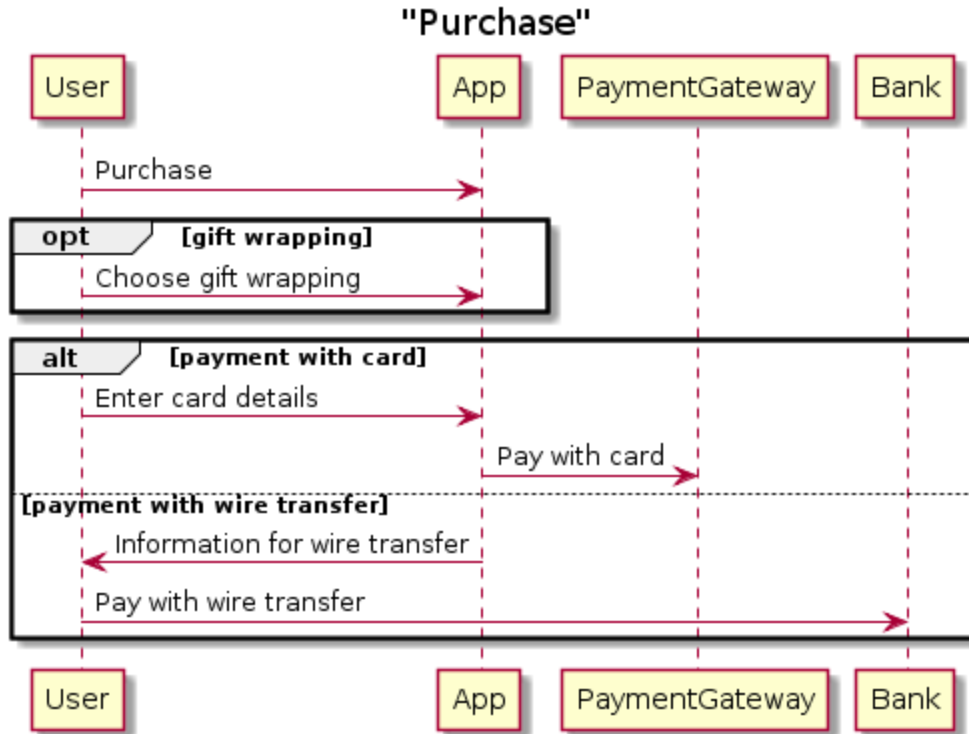
Avoid returns in sequence diagrams unless they add clarity.

Indicating Selection and Loops

- **frame**: box around part of a sequence diagram to indicate selection or loop
 - **if** -> (**opt**) [condition]
 - **if/else** -> (**alt**) [condition], separated by horizontal dashed line
 - **loop** -> (**loop**) [condition or items to loop over]



alt vs. opt



- **alt** is used to describe alternative scenarios of a workflow. Only one of the options will be executed.
- **opt** is used to describe optional step in workflow

More info from UML Specification...

In [UML specification](#) the meaning is described in section 17.12.15. `opt` and `alt` are two different operator types and here's how specification describes them:

`alt`:

The InteractionOperatorKind alt designates that the CombinedFragment represents a choice of behavior. At most one of the operands will be chosen. The chosen operand must have an explicit or implicit guard expression that evaluates to true at this point in the interaction. An implicit true guard is implied if the operand has no guard.

`opt`:

The InteractionOperatorKind opt designates that the CombinedFragment represents a choice of behavior where either the (sole) operand happens or nothing happens. An option is semantically equivalent to an alternative CombinedFragment where there is one operand with non-empty content and the second operand is empty.

<https://www.omg.org/spec/UML/2.5.1/PDF>

Design Sequence Diagram Common Operators

1. Alternative fragment (denoted “alt”)
 - Guard condition specify the true case for the execution of the interaction
 - You can use **[guard condition]** before a message without using ‘alt’
2. Option fragment (denoted “opt”)
 - Guard condition is specified for each case
3. Loop fragment encloses a series of messages which are repeated
 - Guard condition specify the lower and upper limit of the loop
 - You can use ***[guard condition]** to show a loop without using ‘loop’ box/fragment
4. “ref” refers to an interaction defined on another diagram
5. Parallel fragment (denoted “par”) models concurrent processing

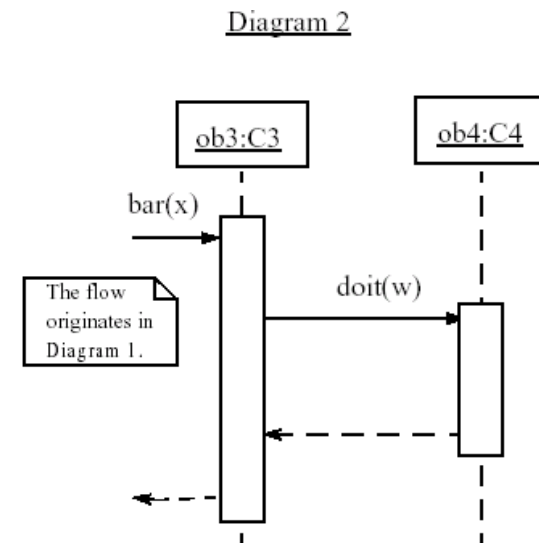
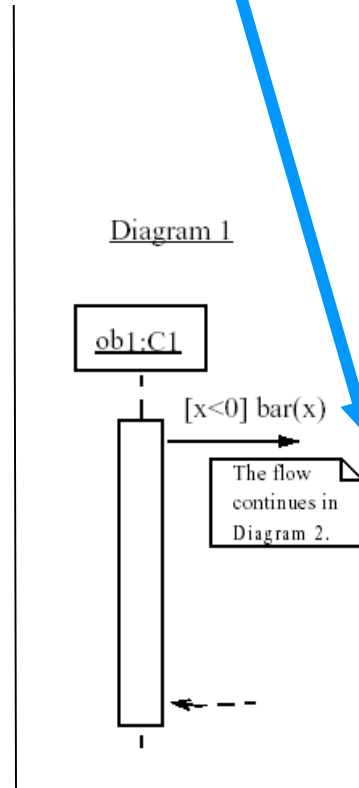
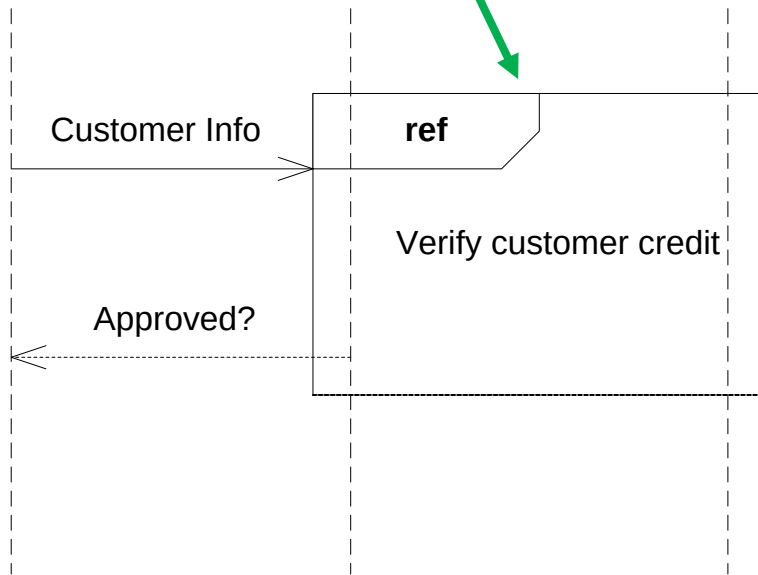
- Check the options for the similar functions (ref, par, loop, alt) in the latest Visual Paradigm Software

Replacing Boxes in the Diagram

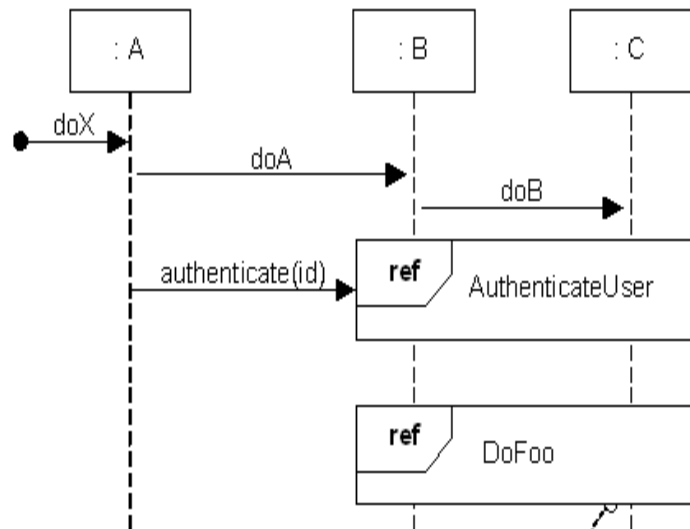
- It is recommended that boxes of **opt**, **alt**, **loop** can be avoided in order to make the diagram simple
- Here are the suggestions to replace boxes
- **opt box** (*if condition..*) can be replaced with the *message with a [guard condition]*
 - Example: 1. **[count < 10]** doIt ();
- **alt box** (*if ...then...else...*) can be replaced with two separate messages; one for the 'true' and the other one is for the 'false' of the condition
 - Example: 1.1 **[count < 10]** doIt ();
1.2 **[count >= 10]** doThat ();
- **loop box** (*repetition*) can be replaced with a * for *while loop*, or a * with guard condition for *FOR loop*
 - Example: 2. ***[total<100]** doIT () //while loop
3. ***[1..10]** doIT () // for loop

Linking Sequence Diagrams

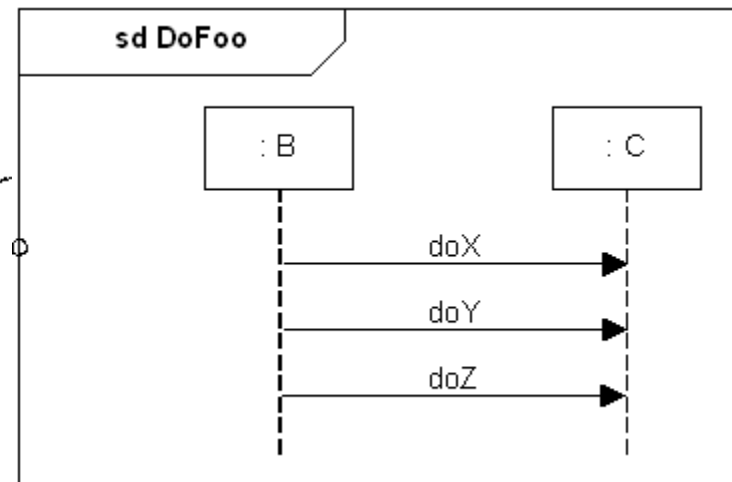
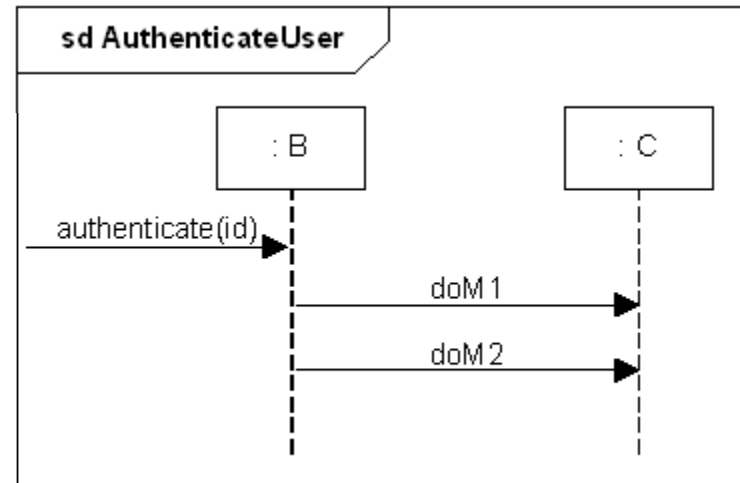
- If one sequence diagram is **too large** or refers to another diagram, indicate it with either
 - an unfinished arrow and comment
 - a "ref" frame that names the other diagram
 - when would this occur in our system?



Referencing Another Design Sequence Diagram (DSD)



interaction occurrence
note it covers a set of lifelines
note that the sd frame it relates to
has the same lifelines: B and C



Modeling Interaction – How-To

- Determine what scenarios need to be modeled
 - Start drawing **one interaction diagram per scenario**
 - If, at some point, the diagram becomes complex, **split your interaction diagram and draw one diagram per system function**
- Identify the **objects that play a role** in the scenario
- Lay the objects out in a sequence diagram left to right, with the **most important** objects to the left
 - **Most important** means objects that are the principle initiators of events
- Draw in the message arrows, top to bottom
 - Each message is assigned a **sequence number** to show the time order of the message
 - **Method calls** should show the **parameters** and the **return type**

Relationship between Design Sequence Diagram and Code (1)

```
public class Cat
{
    private Person owner;

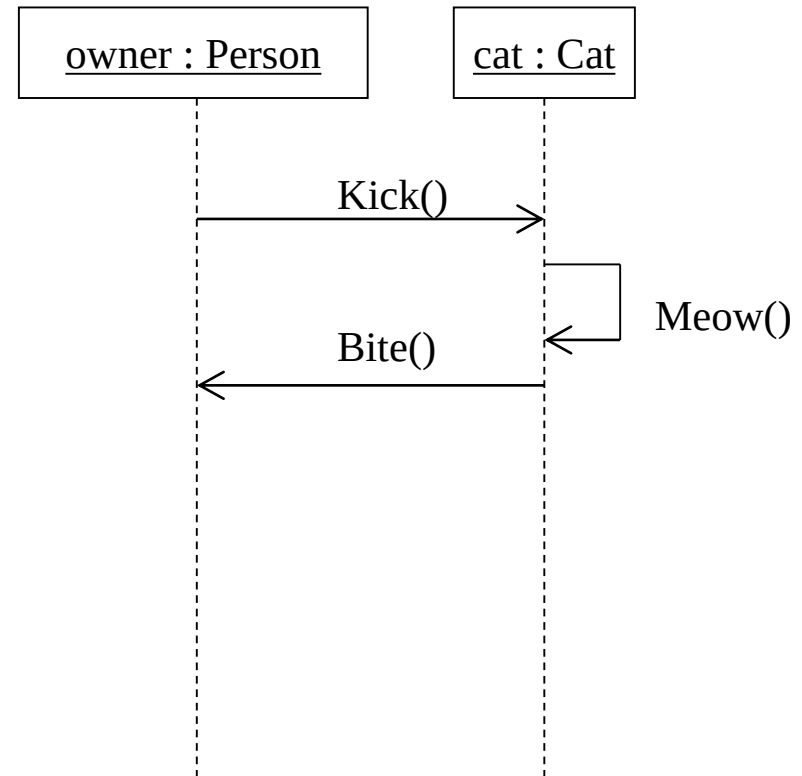
    public void Kick()
    {
        Meow();
        owner.Bite();
    }

    public void Meow()
    {
    }
}

public class Person
{
    private Cat cat;

    public void KickCat()
    {
        // start here
        cat.Kick();
    }

    public void Bite()
    {
    }
}
```



Relationship between Design Sequence Diagram and Code (2)

```
public class Cat
{
    private Person owner;

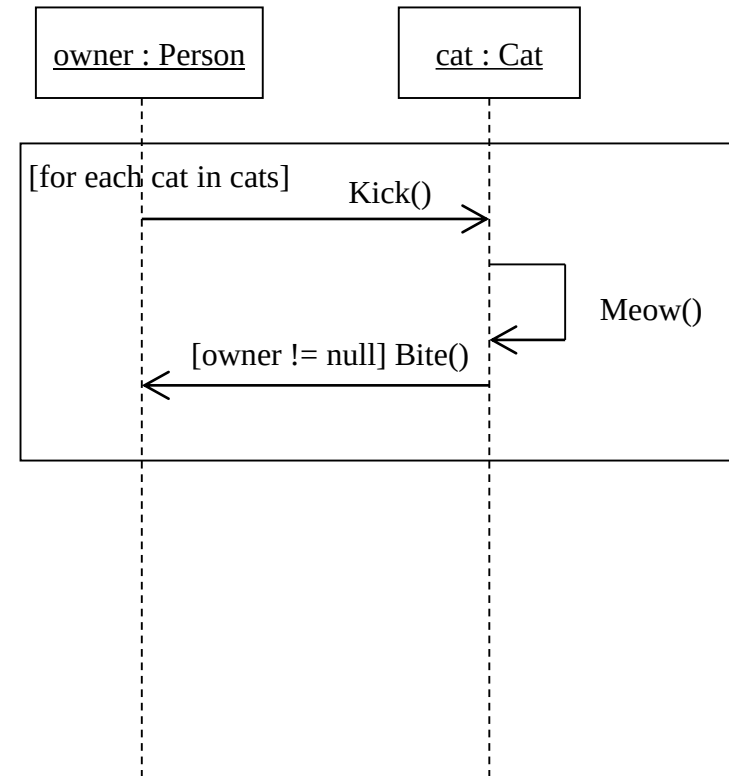
    public void Kick()
    {
        Meow();
        if(owner != null)
        {
            owner.Bite();
        }
    }

    public void Meow()
    {
    }
}
```

```
public class Person
{
    private IList cats;

    public void KickCat()
    {
        // start here
        for each(Cat cat in cats)
        {
            cat.Kick();
        }
    }

    public void Bite()
    {
    }
}
```



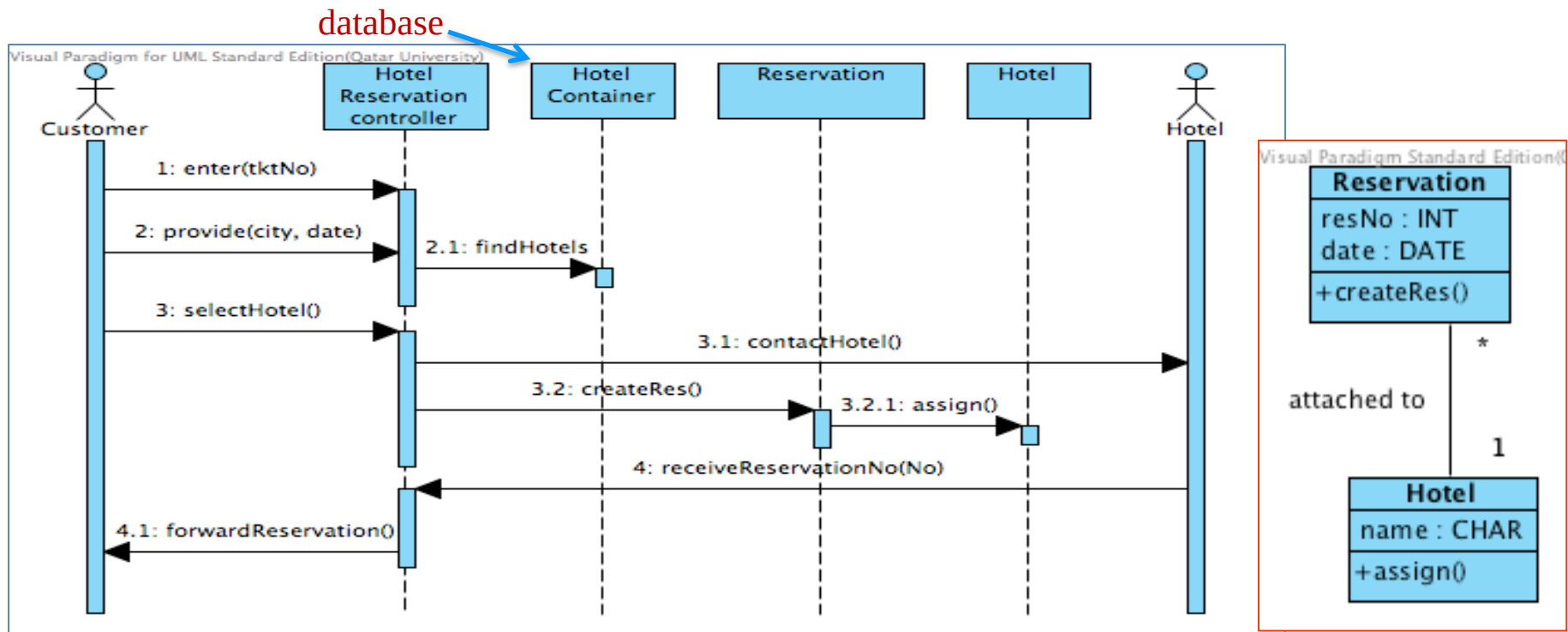
Example: Use case “Book Hotel”

Use case: Book hotel

Actor Action	System Response
1. The customer provides ticket number or booking number	2. Asks for the city, and dates
3. The customer provides the city and dates	4. Find a list of hotel
5. The customer selects the hotel	6. Contacts the hotel
	7. Makes reservation
8. The hotel sends reservation number	9. Forward the reservation information to the customer
10. The customer receives reservation information.	

Use case is transformed to design sequence diagram as shown in the next slide

Sequence Diagram: Book Hotel



- Notice the use of **Controller class** (Boundary class can also be used here)
- Controller class **does not have any method to execute**, it simply acts as an interface between the use case and the outside of the system
- Controller class **does not have any data or methods**
- **Hotel Container** is used as a collection of objects (database). Hotel is an object & an actor
- **Container/collection** class usually executes some pre-defined operations such as find, search, add, delete, etc.
- Container class is **not shown** in the class diagram. Every class has a corresponding container class (database) by default.

Another Example: Modify Seat

Use Case: Modify seat

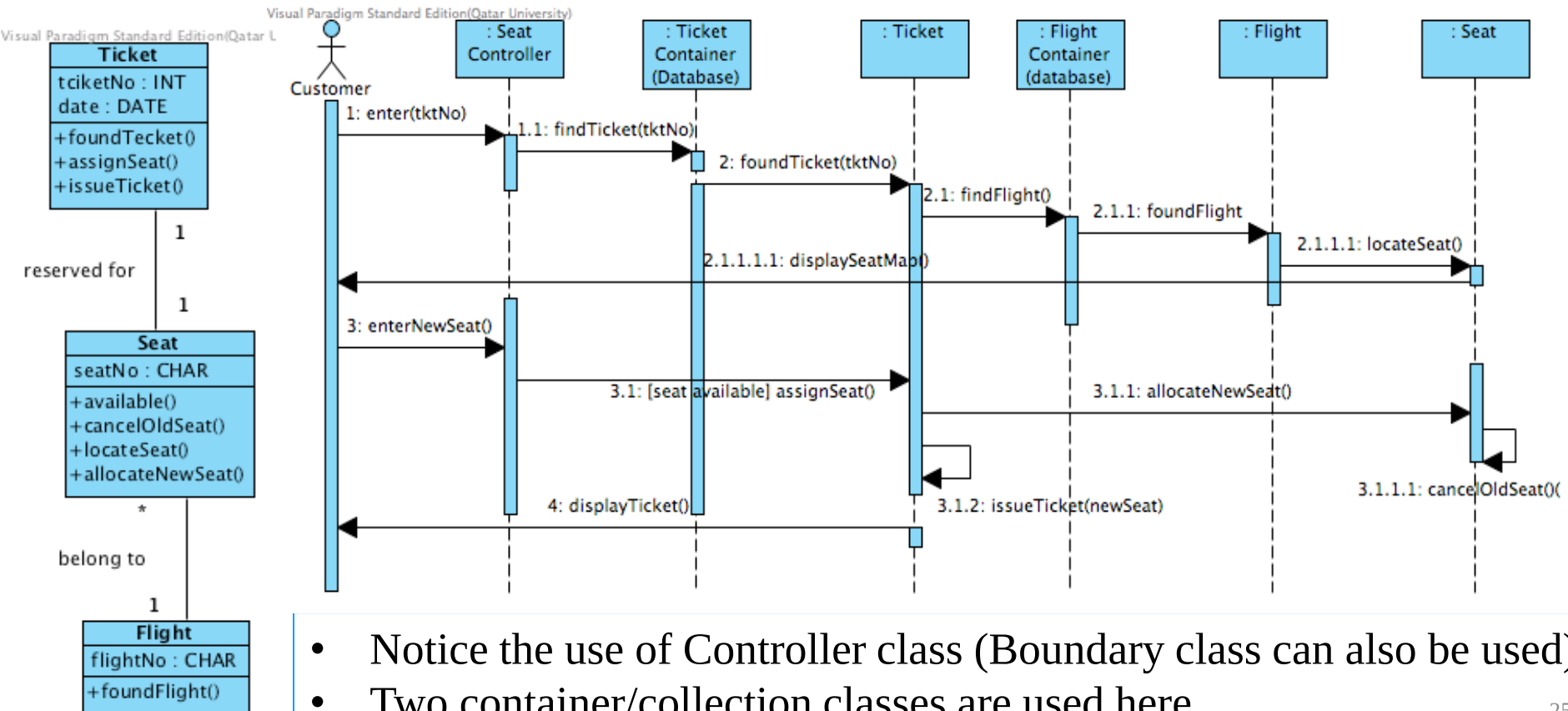
Actor Action	System Response
1. The customer provides the ticket number or booking number	2. Finds the ticket information
	3. Provide the seat map to change the seats if enough seats are available
4. The customer selects new seats	5. Record the new seat information, and release the old seat reservation.
	6. Issue the ticket with new seats

Transform to



Alternative flows:

3a. If enough seats are not available, produce error message.



- Notice the use of Controller class (Boundary class can also be used)
- Two container/collection classes are used here

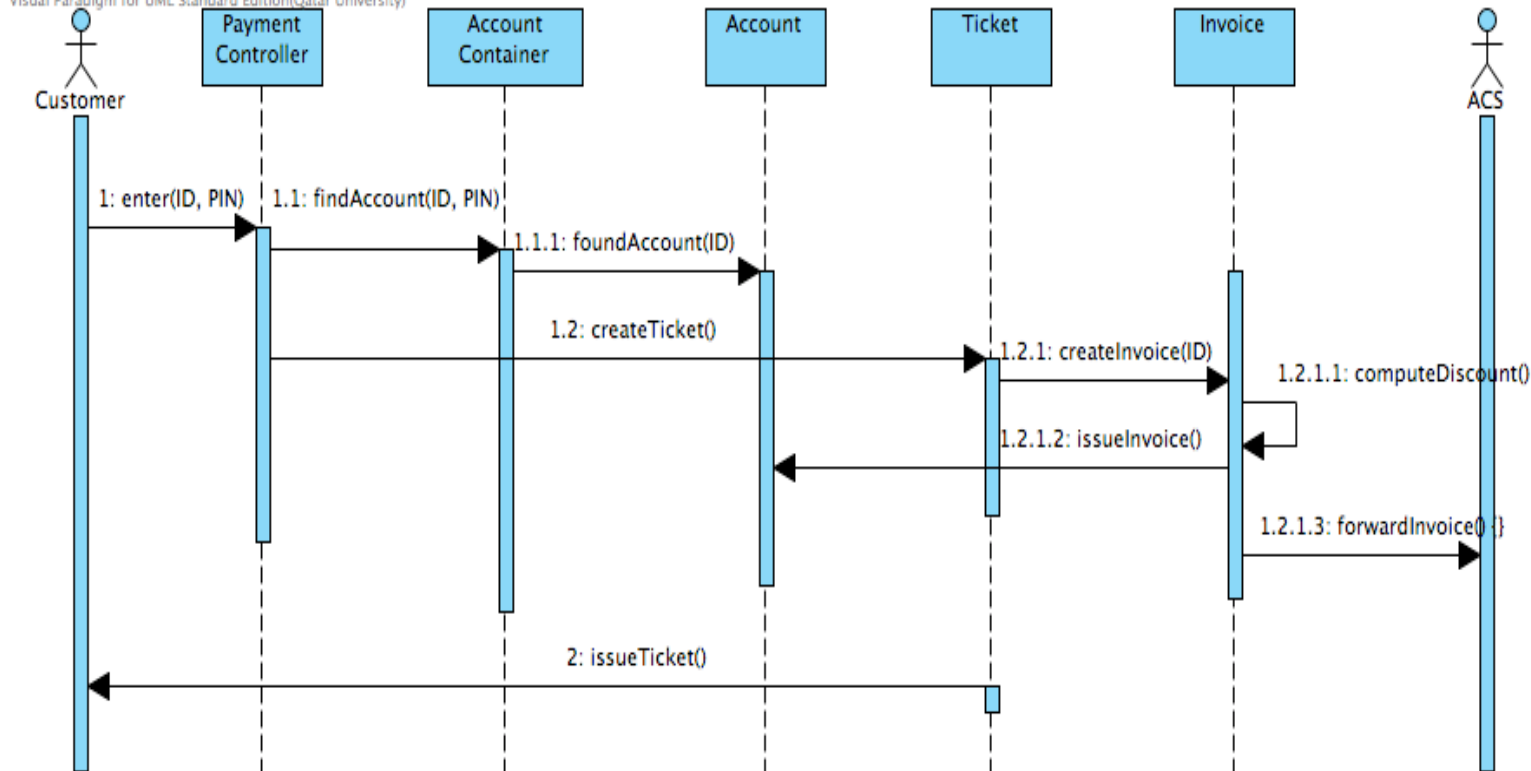
Another Example

Use case: Pay for regular customer

Actor action	System response
1. The customer enters ID and PIN	2. Find the account details
	3. Prepare an invoice
	4. Computes 10% discount
	5. Issue the invoice
	6. Forward the invoice to ACS
	7. Attach the invoice with the account
	8. Prepare and issue ticket.

Transform to

Visual Paradigm for UML Standard Edition(Qatar University)



Rules for Design Sequence Diagram

- ① Objects used in a sequence diagram must have corresponding classes in the class diagram
- ② If an object sends a message to another object in the sequence diagram, there must be an association in the class diagram between the corresponding classes
- ③ If an object sends a message to another object in the sequence diagram, there must be a similar method in the corresponding class of the message receiving object in the class diagram
- ④ If an object calls a reflexive message to itself, the corresponding class must have the same method in the class diagram.
- ⑤ You can use * [guard condition] in front of a message to indicate a loop, instead of using loop box
- ⑥ You can use [guard condition] in front of a message without using “alt” box
- ⑦ Every sequence diagram must have at least one controller class
- ⑧ Controller class does not have any method to execute, it is just an interface and control messages received from/and to actor(s)
- ⑨ A container class is a data storage/database representing the objects of a class. In the class diagram, you do not show these container classes because by default, every entity class in the class diagram has a corresponding database.
- ⑩ Avoid using return message unless it increases clarity

Do not make your sequence diagram unnecessarily complex using too many boxes and return messages

References

- <https://www.visual-paradigm.com/support/documents/vpuserguide/94/2577/7025drawingseque.html>
- <https://creately.com/blog/diagrams/sequence-diagram-tutorial/>